HW2 — Temporal logic and model checking

CS 3892 / CS 5892 · Formal Methods: Safe and Trustworthy AI · Fall 2026

Out	Thursday, October 1
Due	**Tuesday, October 13, 11:59 p.m. US Central**
Weight	4% of the course grade (100 points)
Tool	nuXmv, or NuSMV — the same SMV language, and the same verdicts on every model here
Zero-install path	smvis in the browser, or NuSMV in Google Colab or GitHub Codespaces
Submit	Brightspace → Assignments → *HW2 — Temporal logic and model checking (nuXmv)*

Nothing to install. Every part of this runs in a browser tab.

Start here: github.com/ttj/cs3892-examples has the four starter models for this homework in `homework/hw2-temporal-logic-model-checking/smv/`, one for each of Parts 1–4, each with its variables documented in its header.

What you need, and when you get it. Thursday Oct 1 (LTL: `G`, `F`, `X`, `U`; safety and liveness) covers Parts 1, 3 and 5 and seven of Part 2's eight requirements. **Requirement R8 of Part 2 needs CTL's path quantifiers — `EF` versus `AF` — which is Tuesday Oct 6's material.** Part 1's one `SPEC` line uses `AG p`, which you have been reading as "`p` holds in every reachable state" since Sep 24; that is all it needs. Part 4 is bounded model checking, which you met with Z3 on Sep 22. nuXmv's BMC engine is Tuesday Oct 13's lecture — the day HW2 is due — so every command Part 4 needs is given below, and you do not need that lecture to do it.

What this is for

By the end you should be able to read a model the way the model checker reads it, write an English requirement as a temporal formula, and — the part that matters — **know whether the verdict you got answers the question you meant to ask.** A model checker decides exactly the formula you wrote, about exactly the model you gave it. `true` can mean the requirement holds; it can also mean you wrote a weaker formula than the requirement, or that the model has fewer behaviours than the system it stands for. And a bounded check that finds nothing has only looked as far as its bound.

Running the models

In the browser. smvis runs NuSMV on your own `.smv` files and draws the state graph — enough to check properties and read counterexamples. Part 1's `-r` count and all of Part 4 use NuSMV command-line options, so for those use one of the routes below.

In Codespaces. The course container described in `labs/README.md` has `NuSMV` preinstalled. Clone the examples repository into it and run `NuSMV` directly.

In Colab, or any Linux shell. NuSMV is free and ships as a static Linux binary. In a Colab cell, put `!` in front of each line:

```
git clone --depth 1 https://github.com/ttj/cs3892-examples.git
mkdir -p nusmv
curl -sSLO https://nusmv.fbk.eu/distrib/NuSMV-2.6.0-linux64.tar.gz
tar -xzf NuSMV-2.6.0-linux64.tar.gz -C nusmv --strip-components=1
cp -r cs3892-examples/homework/hw2-temporal-logic-model-checking/smv hw2
nusmv/bin/NuSMV hw2/agent_loop.smv
```

`hw2/` is now your own working copy of the four starters.

Locally. NuSMV from nusmv.fbk.eu, or nuXmv from nuxmv.fbk.eu, which is licence-gated. NuSMV is enough for every part.

`NuSMV file.smv` checks every property in the file and prints one verdict line per property, with a counterexample under each one that is false. It does not always print them in the order of the file — every CTL (`SPEC`) result comes before every LTL (`LTLSPEC`) one, for a start — so **match each verdict to its property by the formula it prints, not by its position.**

Part 1 — Read a model *(15 points)*

`agent_loop.smv` is a tool-using agent reduced to its control loop: it idles until a task arrives, makes three tool calls, reports, and goes back to idle. Whether a task arrives is a *free input* that the model checker may choose at every step. Four properties, `P1`–`P4`, sit at the bottom of the file.

Before you run anything, write down:

1. the initial state or states — all of them;
2. the transition relation, as a table or a diagram: from each state, which states can follow it;
3. the size of the state space, two ways: how many states the declarations allow, and how many are actually reachable from the initial states;
4. your predicted verdict for each of `P1`–`P4`, with a sentence of reasoning, and — for each one you predict false — the counterexample you expect.

Then run it. `NuSMV -r agent_loop.smv` also prints the number of reachable states; use it to check your count, not to produce it. **Report every place your prediction was wrong, and what you had misread.** A wrong prediction you can explain is worth more here than a right one you cannot.

Part 2 — Write specifications *(30 points)*

`traffic_light.smv` is a vehicle-actuated traffic light. A main road and a side road share an intersection; the main road is green by default, and a detector on the side road decides when the side road gets a turn. The model is complete. **The specifications are yours.**

The model. One step is one controller tick.

Name	Values	Meaning
`phase`	`MG`, `MY`, `AR_S`, `SG`, `SY`, `AR_M`	where the controller is in its cycle: main green, main yellow, all red with the side road next, side green, side yellow, all red with the main road next

Name	Values	Meaning
`timer`	`0..2`	ticks already spent in the current green phase
`car`	`TRUE`, `FALSE`	**free input** — the side-road detector sees a vehicle this tick. It has no `init` and no `next`, so its value is chosen afresh at every step, *including the first*
`call`	`TRUE`, `FALSE`	a side-road vehicle has been detected and not yet served. Starts `FALSE`; set by `car`; cleared by the side road's green phase
`main_green` … `side_red`	`TRUE`, `FALSE`	six abbreviations (`DEFINE`), one per lamp: `main_green`, `main_yellow`, `main_red`, `side_green`, `side_yellow`, `side_red`

The eight requirements. Formalize each one as a single LTL or CTL formula over the names above, add it to the file (`LTLSPEC` for LTL, `SPEC` for CTL) under the comment carrying its number, and run it.

- **R1.** The main road and the side road are never shown green at the same time.
- **R2.** A detected vehicle is never forgotten: once `call` is true, it stays true until the side road's light is green.
- **R3.** The main road gives up its green only on demand: whenever the main road's light changes from green to yellow, `call` is true at the moment of the change.
- **R4.** Every time the main road's light turns green, it stays green for at least three consecutive steps.
- **R5.** Every time the side road's light turns green, it stays green for at least two consecutive steps.
- **R6.** Every vehicle the side-road detector sees eventually gets a green light.
- **R7.** However the side-road traffic behaves, the main road gets its green back — again and again, forever.
- **R8.** Whatever has happened so far, the controller can still give the side road a green light: the side road is never shut out for good. (It is not *promised* one. If no vehicle ever comes, it never needs one.)

For each requirement, report the formula, whether it is LTL or CTL, and the verdict. For R1–R7, also say whether the requirement is a safety or a liveness property, with a one-line reason.

For R8 — Tuesday Oct 6's material — also check the formula you get by replacing `EF` with `AF` in yours (or `AF` with `EF`, if that is what you wrote). Report both verdicts and explain, in two or three sentences, why they differ on this model. Point at the run that makes the difference.

The model does not necessarily satisfy every requirement. When one of your formulas comes back false, the question is whether your formula mistranslates the requirement or the model violates it. Say which, and use the counterexample to show it. Editing a formula until it comes back true is not the exercise.

Part 3 — Break it *(20 points)*

`counter.smv` is the course's running example, exactly as **`labs/running-example.md`** defines it: `x` starts at 0, adds one while `x < 10`, and resets to 0 at the cap. The property `x ≤ 10` holds, and both properties in the file — the same property written once as an invariant and once in LTL — come back true.

`x` is declared `0..25`, not over all the integers, because NuSMV needs every type to be finite. It is wider than `0..10` on purpose: with `0..10`, the property would hold by declaration and there would be nothing to check.

1. **Plant a bug.** Change `next(x)` — nothing else — so that `x ≤ 10` fails. Obtain both counterexamples, the invariant's and the LTL one.

2. **Explain the invariant's counterexample step by step.** For each state in the trace, say which branch of the `case` fired to produce the next state. Then say at which state the property fails, and why there. NuSMV labels each state `t.n`: `t` numbers the counterexamples in a run (so the invariant's may well be trace 2), and `n` counts states from 1, so `t.1` is the initial state, reached after zero steps. Give the number of *transitions* after which `x` first breaks the property.
3. **Compare the two counterexamples.** The LTL one carries a `-- Loop starts here` marker that the invariant's lacks. Say why a counterexample to an LTL formula takes that shape, and why the invariant's counterexample can stop where it does.
4. **Plant a second, different bug that `x ≤ 10` does not catch.** Make `next(x)` behave differently from the canonical machine while both properties still come back true. Then write one more property that does catch it, and show that it does.

Keep your arithmetic inside `0..25`. NuSMV checks every branch of `next(x)` against the declared type, for every declared value of `x`, reachable or not. If any branch can produce a value outside it, the run stops with `cannot assign value 26 to variable x` before checking anything. That is a type error, not a counterexample.

Part 4 — Bound hunt *(20 points)*

`bmc_depth.smv` hides a bug: a state in which `bug` is true, which the property `G !bug` says never happens. The ordinary check, `NuSMV bmc_depth.smv`, uses BDDs, looks at every reachable state, and reports the property false. Bounded model checking looks only as far as you tell it to:

```
NuSMV -bmc -bmc_length K bmc_depth.smv
```

This checks every run of at most `K` steps. It prints `no counterexample found with bound j` for each bound `j` from 0 up to `K` that it clears, and stops at the first bound where it finds a counterexample. Leave out `-bmc_length` and `K` is 10. nuXmv takes the same command line.

1. **Find the smallest bound that exposes the bug.** Show the output at that bound and at the bound one below it.
2. **Say precisely what the bound counts.** Your counterexample lists states `1.1`, `1.2`, and so on. How many states does it list, and how does that number relate to the bound? Say which state is the initial state and after how many transitions the bug is reached. Get this exactly right: an off-by-one here is the commonest way to misreport a BMC result.
3. **Explain why that bound, from the model itself.** Give an argument about the two counters' arithmetic that predicts the depth without running anything — as in HW1 Part 3, "the tool said so" is not the explanation.
4. **Say what a passing run at bound 10 would have told you, and what it would not.** That is exactly the run `NuSMV -bmc bmc_depth.smv` gives you, since 10 is the default. Then: how large a bound would a BMC run on this model need before "no counterexample" could be read as a proof that an invariant holds, and why? `NuSMV -r bmc_depth.smv` prints a "system diameter". Work out what that number counts before you rely on it.

Part 5 — AI component *(15 points)*

Have an LLM write **LTL** for the same eight requirements. Give it the requirement text and the variable table, or the model file, and nothing of your own. Then **check its formulas mechanically against yours, not by eye:**

- **On the model.** For each requirement, add `LTLSPEC (yours) <-> (theirs)` to the model and run it. If it is true, the two formulas agree on every run *of this model*. If it is false, the counterexample is a run on which they disagree: read it, and say which formula is right about that run.
- **In general.** Agreement on the model is not equivalence — the fewer behaviours a model has, the more formulas it makes agree. For every pair that agrees on the model, run the same check on a copy of `traffic_light.smv` with its whole `ASSIGN` section deleted, keeping `VAR` and `DEFINE`. With no `ASSIGN`, every variable is free at every step, so every sequence of values is a run, and `<->` is checked against all of them. Report the pairs that agree on the model but not in general, and say whether the difference matters for this controller.
- **Where your formula is CTL**, `<->` cannot join it to an LTL formula. Run the two separately, compare their verdicts, and say whether that requirement can be written in LTL at all.

When two formulas are equivalent, NuSMV sometimes prints a `WARNING` that "the initial states set of the finite state machine is empty" just before the verdict. The verdict line after it is the result.

Report the prompt you used, the LLM's eight formulas, a table with one row per requirement (agrees on the model? agrees in general? who was right?), and a paragraph on what kinds of mistake it made.

What to submit

One archive or repository link containing:

1. Your `.smv` files, runnable as-is: `traffic_light.smv` with your eight specifications, your two buggy copies of `counter.smv` (each with its extra property, if it has one), and the files you used for Part 5.
2. A short report — Markdown or PDF, roughly three pages — with your Part 1 predictions and what happened, the Part 2 table, the Part 3 trace walk-through, the Part 4 answers, and your Part 5 write-up.
3. The AI-use disclosure (see below).

The mechanical part is checked automatically: do your files run, and do they produce the verdicts you claim? The interesting part — why the counterexample looks like that — is read by a human.

Late policy

Four late days for the semester, in whole-day units, no more than two on any one homework, no questions asked. Beyond that, 20% per day.

Collaboration and AI use

Discussion is encouraged; submitted work must be your own. **AI assistants are allowed and expected** — this is a course about the reliability of AI-produced artifacts. Two conditions:

1. **Disclose.** Say what you used and for what.
2. **Verify.** You are responsible for everything you submit, and "the model said so" does not transfer that responsibility. It carries less weight in this course than almost anywhere else, because every tool you will use is a decision procedure that can tell you whether the model was right — so use one.